



FastAPI 是一个用于构建 API 的现代、快速 (高性能) 的 web 框架, 专为在 Python 中构建 RESTful API 而设计。

FastAPI 使用 Python 3.8+ 并基于标准的 Python 类型提示。

FastAPI 建立在 Starlette 和 Pydantic 之上, 利用类型提示进行数据处理, 并自动生成 API 文档。

FastAPI 于 2018 年 12 月 5 日发布第一版本, 以其易用性、速度和稳健性在开发者中间迅速流行起来。

FastAPI 支持异步编程, 可在生产环境中运行。

谁适合阅读本教程?

本教程适合有 Python 基础的开发者学习。

学习本教程前你需要了解

学习本教程前你需要了解一些基础的 Web 知识及 [Python 3.x 基础教程](#)。

FastAPI 特点

- **高性能:** 基于 Starlette 和 Pydantic, 利用异步 (asynchronous) 编程, 提供出色的性能。
- **自动文档生成:** 自动生成交互式 API 文档, 支持 Swagger UI 和 ReDoc, 让 API 的理解和测试更加直观。
- **类型注解支持:** 利用 Python 的类型提示, 提供更严格的输入验证和更好的代码提示。
- **异步支持:** 支持异步请求处理, 使得处理 IO 密集型任务更加高效。

FastAPI 适用场景

- **构建 API 后端:** 用于构建 RESTful API, 支持前后端分离的 Web 应用。
- **微服务架构:** 可以作为微服务的后端框架, 支持快速开发和部署。
- **数据处理 API:** 适用于处理数据, 接收和返回 JSON 数据。
- **实时通信:** 支持 WebSocket, 适用于实时通信场景。

为什么选择 FastAPI?

- **Pythonic:** 使用 Python 的自然语法和类型提示, 降低学习曲线。
- **性能优越:** 利用异步编程和底层的 Starlette 框架, 提供卓越的性能。
- **文档友好:** 自动生成交互式文档, 减少文档维护的工作量。
- **生态系统:** 基于 Python 生态系统, 可以方便地集成各种库和工具。

相关链接

- FastAPI 文档: <https://fastapi.tiangolo.com/zh/>
- FastAPI 源码: <https://github.com/tiangolo/fastapi>

FastAPI 安装

FastAPI 依赖 Python 3.8 及更高版本。

安装 FastAPI 很简单，这里我们使用 **pip** 命令来安装。

```
pip install fastapi
```

另外我们还需要一个 ASGI 服务器，生产环境可以使用 Uvicorn 或者 Hypercorn：

```
pip install "uvicorn[standard]"
```

这样我们就安装完成了。

运行第一个 FastAPI 应用

创建一个名为 `main.py` 的文件，添加以下代码：

实例

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def read_root():
```

```
    return {"Hello": "World"}
```

在命令行中运行以下命令以启动应用：

```
uvicorn main:app --reload
```

现在，打开浏览器并访问 **http://127.0.0.1:8000**，你应该能够看到 FastAPI 自动生成的交互式文档，并在根路径 ("/") 返回的 JSON 响应。

```
1 // 20231219103752
2 // http://127.0.0.1:8000/
3
4 ▾ {
5     "Hello": "World"
6 }
```

代码解析：

- **from fastapi import FastAPI**：这行代码从 `fastapi` 模块中导入了 `FastAPI` 类。`FastAPI` 类是 FastAPI 框架的核心，用于创建 FastAPI 应用程序实例。
- **app = FastAPI()**：这行代码创建了一个 FastAPI 应用实例。与 Flask 不同，FastAPI 不需要传递 `__name__` 参数，因为它默认使用当前模块。
- **@app.get("/")**：这是一个装饰器，用于告诉 FastAPI 哪个 URL 应该触发下面的函数，并且指定了 HTTP 方法为 GET。在这个例子中，它指定了根 URL（即网站的主页）。
- **def read_root():**：这是定义了一个名为 `read_root` 的函数，它将被调用当用户使用 GET 方法访问根 URL 时。
- **return {"Hello": "World"}**：这行代码是 `read_root` 函数的返回值。当用户使用 GET 方法访问根 URL 时，这个 JSON 对象将被发送回用户的浏览器或 API 客户端。

FastAPI 教程

第一个 FastAPI 应用

创建一个名为 `main.py` 的文件，添加以下代码：

实例

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def read_root():
```

```
    return {"Hello": "World"}
```

在命令行中运行以下命令以启动应用：

```
uvicorn main:app --reload
```

现在，打开浏览器并访问 **`http://127.0.0.1:8000`**，你应该能够看到 FastAPI 自动生成的交互式文档，并在根路径（`/`）返回的 JSON 响应。

```
1 // 20231219103752
2 // http://127.0.0.1:8000/
3
4 ▼ {
5     "Hello": "World"
6 }
```

接下来我们来丰富下代码功能，并做具体说明。

以下的 FastAPI 应用，使用了两个路由操作（`/` 和 `/items/{item_id}`）：

实例

```
from typing import Union
```

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def read_root():
```

```
    return {"Hello": "World"}
```

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int, q: Union[str, None] = None):
```

```
    return {"item_id": item_id, "q": q}
```

代码拆解分析

1、导入必要的模块和类：

```
from typing import Union
```

```
from fastapi import FastAPI
```

FastAPI 教程

以上实例导入了 Union 类型，用于支持多种数据类型的参数注解。

2、创建 FastAPI 实例：

```
app = FastAPI()
```

在这一步，创建了一个 FastAPI 应用的实例，它将用于定义和管理应用的各个组件，包括路由。

FastAPI 是 FastAPI 框架的主要类。

3、定义根路径 / 的路由操作：

```
@app.get("/")
```

```
def read_root():
```

```
    return {"Hello": "World"}
```

这个路由操作使用了 `@app.get("/")` 装饰器，表示当用户通过 HTTP GET 请求访问根路径时，将执行 `read_root` 函数。函数返回一个包含 `{"Hello": "World"}` 的字典，这个字典会被 FastAPI 自动转换为 JSON 格式并返回给用户。

4、定义带路径参数和查询参数的路由操作：

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int, q: Union[str, None] = None):
```

```
    return {"item_id": item_id, "q": q}
```

这个路由操作使用了 `@app.get("/items/{item_id}")` 装饰器，表示当用户通过 HTTP GET 请求访问 `/items/{item_id}` 路径时，将执行 `read_item` 函数。

函数接受两个参数：

- `item_id` -- 是路径参数，指定为整数类型。
- `q` -- 是查询参数，指定为字符串类型或空 (None)。

函数返回一个字典，包含传入的 `item_id` 和 `q` 参数。

`q` 参数通过 `Union[str, None]` 表示可以是字符串类型或空，这样就允许在请求中不提供 `q` 参数。

使用浏览器访问 `http://127.0.0.1:8000/items/5?q=runoob`，你将会看到如下 JSON 响应：

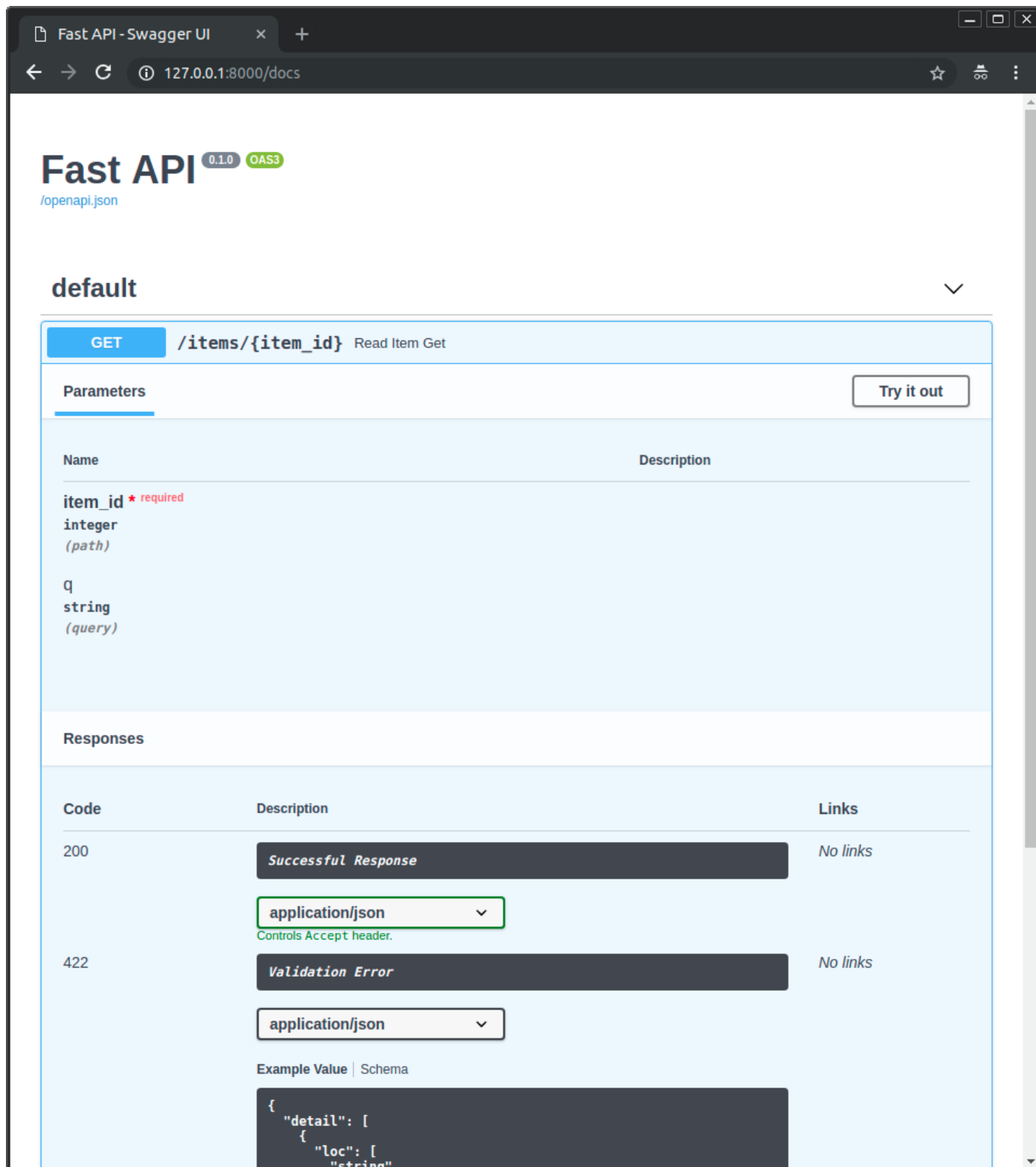
```
{"item_id": 5, "q": "runoob"}
```

如下图所示：

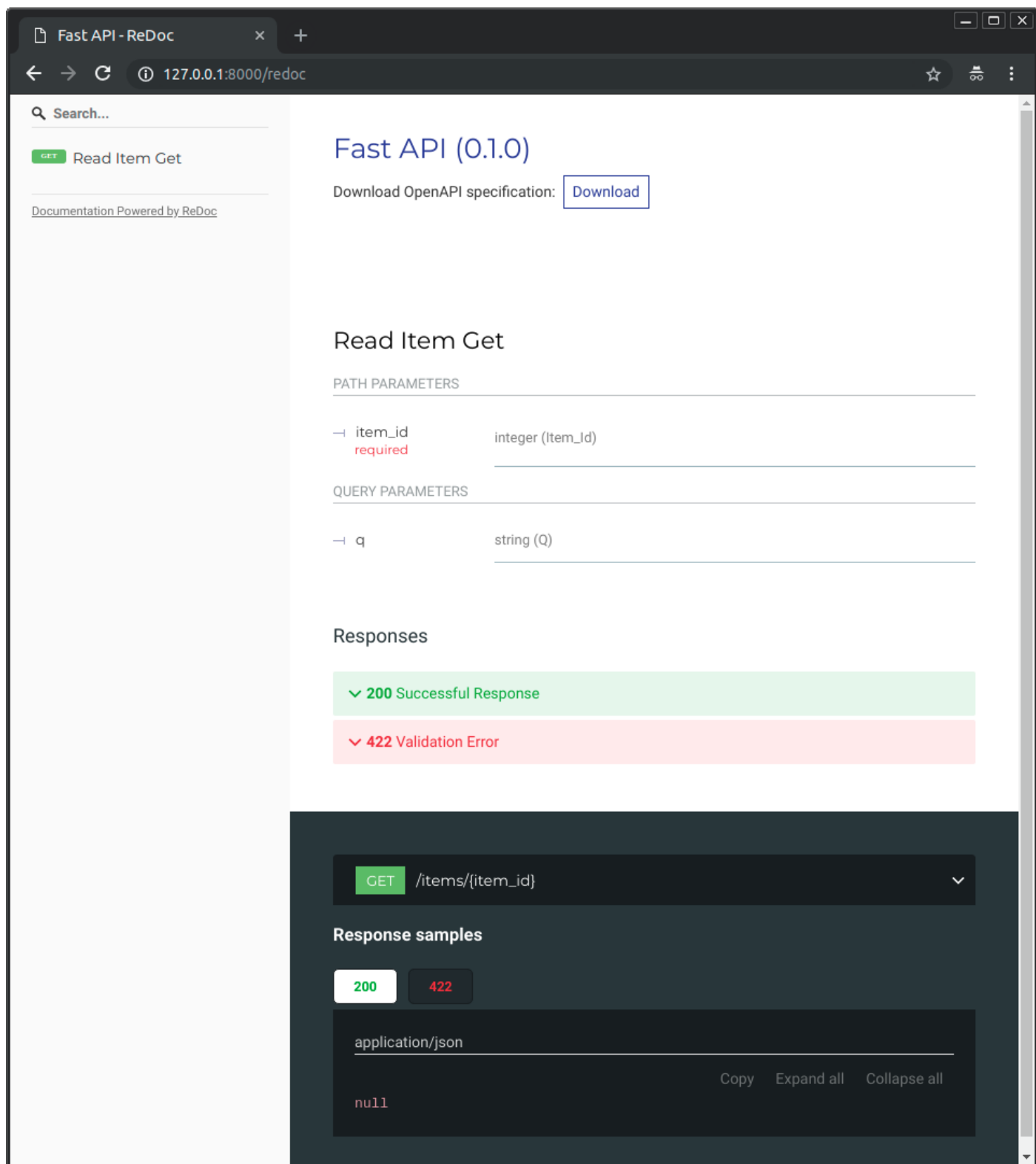
```
1 // 20231219105520
2 // http://127.0.0.1:8000/items/5?q=runoob
3
4 {
5     "item_id": 5,
6     "q": "runoob"
7 }
```

FastAPI 交互式 API 文档

FastAPI 提供了内置的交互式 API 文档，使开发者能够轻松了解和测试 API 的各个端点。这个文档是自动生成的，基于 OpenAPI 规范，支持 Swagger UI 和 ReDoc 两种交互式界面。通过 FastAPI 的交互式 API 文档，开发者能够更轻松地理解和使用 API，提高开发效率。在运行 FastAPI 应用时，Uvicorn 同时启动了交互式 API 文档服务。默认情况下，你可以通过访问 <http://127.0.0.1:8000/docs> 来打开 Swagger UI 风格的文档：



Swagger UI 提供了一个直观的用户界面，用于浏览 API 的各个端点、查看请求和响应的结构，并支持直接在文档中进行 API 请求测试。通过 Swagger UI，你可以轻松理解每个路由作的输入参数、输出格式和请求示例。或者通过 <http://127.0.0.1:8000/redoc> 来打开 ReDoc 风格的文档。



ReDoc 是另一种交互式文档界面，具有清晰简洁的外观。它使得开发者能够以可读性强的方式查看 API 的描述、请求和响应。与 Swagger UI 不同，ReDoc 的设计强调文档的可视化和用户体验。

交互式文档的优势

- **实时更新：** 交互式文档会实时更新，反映出应用代码的最新更改。
- **自动验证：** 输入参数的类型和格式会得到自动验证，降低了错误的可能性。
- **便于测试：** 可以直接在文档中进行 API 请求测试，避免使用其他工具。

升级实例

接下来我们修改 `main.py` 文件来从 PUT 请求中接收请求体。

我们借助 Pydantic 来使用标准的 Python 类型声明请求体。

实例

FastAPI 教程

```
from typing import Union
```

```
from fastapi import FastAPI
```

```
from pydantic import BaseModel
```

```
app = FastAPI ()
```

```
class Item (BaseModel):
```

```
    name: str
```

```
    price: float
```

```
    is_offer: Union[bool, None] = 无
```

```
@app.get ("/")
```

```
def read_root ():
```

```
    return {"Hello": "World"}
```

```
@app.get ("/items/{item_id}")
```

```
def read_item (item_id: int, q: Union[str, None] = None):
```

```
    返回 {"item_id": item_id, "q": q}
```

```
@app.put ("/items/{item_id}")
```

```
def update_item (item_id: int, item: Item):
```

```
    return {"item_name": item.名称 "item_id": item_id}
```

修改完代码后，我们不需要重启服务器，因为服务器将会自动重载，因为在前面的章节中 **uvicorn** 命令添加了 **--reload** 选项。

这时候我们再访问 **http://127.0.0.1:8000/docs**，交互式 API 文档将会自动更新，并加入新的请求体：

The screenshot shows the FastAPI Swagger UI in a web browser. The browser's address bar displays '127.0.0.1:8000/docs'. The page title is 'Fast API' with version '0.1.0' and 'OAS3' tags. Below the title, there's a 'default' tab. The main content area shows a list of endpoints. The selected endpoint is 'PUT /items/{item_id} Save Item Put'. It has a 'Parameters' section with a table showing 'item_id' as a required integer path parameter. There is a 'Request body' section marked as 'required' with a dropdown menu set to 'application/json'. Below this, there's an 'Example Value' section showing a JSON object: {'name': 'string', 'price': 0, 'is_offer': true}. At the bottom, there's a 'Responses' section with a table showing a '200' status code with the description 'Successful Response' and 'No links'.

Fast API 0.1.0 OAS3
/openapi.json

default

GET / Read Root Get

GET /items/{item_id} Read Item Get

PUT /items/{item_id} Save Item Put

Parameters Try it out

Name	Description
item_id ★ required	
integer	
(path)	

Request body required application/json

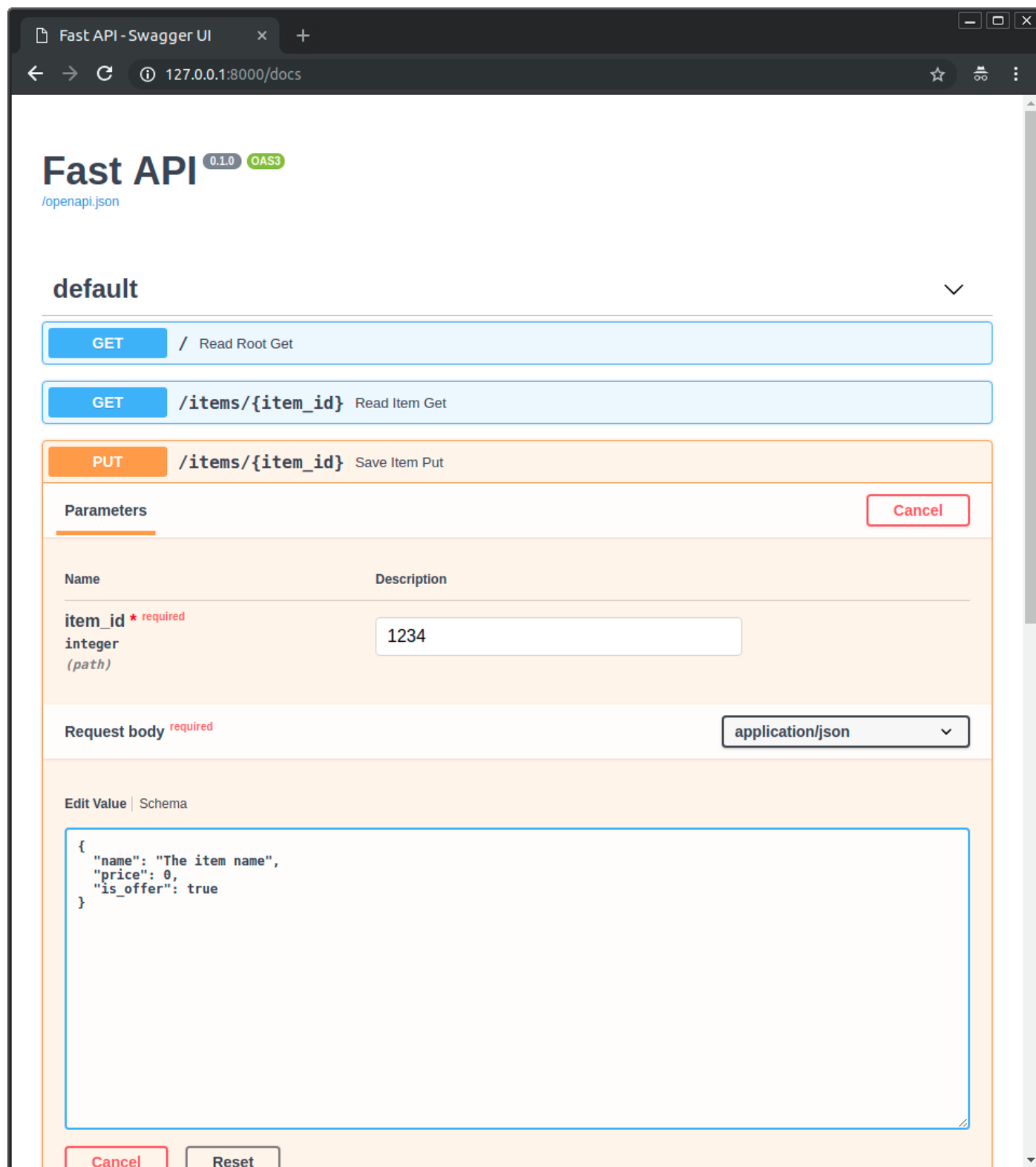
Example Value | Schema

```
{
  "name": "string",
  "price": 0,
  "is_offer": true
}
```

Responses

Code	Description	Links
200	Successful Response	No links

点击「Try it out」按钮，之后你可以填写参数并直接调用 API：



然后点击「Execute」按钮，用户界面将会和 API 进行通信，发送参数，获取结果并在屏幕上展示：

FastAPI 教程

The screenshot shows the FastAPI Swagger UI in a web browser. The address bar displays `127.0.0.1:8000/docs`. The interface has a dark theme. At the top, there's a tab labeled "Fast API - Swagger UI". Below the address bar, there are navigation icons. The main content area is divided into two sections: "Edit Value" and "Schema". The "Edit Value" section contains a JSON body:

```
{  "name": "The item name",  "price": 0,  "is_offer": true}
```

. Below this, there are "Cancel" and "Reset" buttons. Further down, there are "Execute" and "Clear" buttons. The "Responses" section is expanded, showing the "Curl" command:

```
curl -X PUT "http://127.0.0.1:8000/items/1234" -H "accept: application/json" -H "Content-Type: application/json" -d '{"name":"The item name","price":0,"is_offer":true}'
```

. Below the curl command, the "Request URL" is `http://127.0.0.1:8000/items/1234`. The "Server response" section shows a "200" status code. The "Response body" is

```
{  "item_name": "The item name",  "item_id": 1234}
```

. There is a "Download" button next to the response body. The "Response headers" section shows `content-length: 44` and `content-type: application/json`.

另外，访问 `http://127.0.0.1:8000/redoc`，可选文档同样会体现新加入的请求参数和请求体：

Fast API - ReDoc

Search...

GET Read Root Get

GET Read Item Get

PUT Save Item Put

Documentation Powered by ReDoc

Save Item Put

PATH PARAMETERS

→ item_id required	integer (Item_Id)
-----------------------	-------------------

REQUEST BODY SCHEMA: application/json

name required	string (Name)
price required	number (Price)
is_offer	boolean (Is_Offer)

Responses

- ✓ 200 Successful Response
- ✓ 422 Validation Error

PUT /items/{item_id}

Request samples

Payload

application/json

```
{
  "name": "string",
  "price": 0,
  "is_offer": true
}
```

Copy Expand all Collapse all

FastAPI 教程

FastAPI 基本路由

在 FastAPI 中，基本路由是定义 API 端点的关键。每个路由都映射到应用程序中的一个函数，用于处理特定的 HTTP 请求，并返回相应的响应。

根路径路由

创建 FastAPI 实例和根路径路由：

实例

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
def read_root():
    return {"Hello": "World"}
```

代码说明：

- FastAPI(): 创建 FastAPI 应用实例。
- @app.get("/"): 使用 @app.get 装饰器创建一个处理根路径的路由。
- def read_root(): 路由处理函数，返回一个包含 {"Hello": "World"} 的字典。

路径参数

设置路由的参数：

实例

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
def read_root():
    return {"Hello": "World"}
```

```
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str = None):
    return {"item_id": item_id, "q": q}
```

代码说明：

- @app.get("/items/{item_id}"): 定义了一个路由路径，其中 {item_id} 是路径参数，对应于函数参数 item_id。
- def read_item(item_id: int, q: str = None): 路由处理函数接受一个整数类型的路径参数 item_id 和一个可选的字符串类型查询参数 q。

在路由操作中，可以使用函数参数声明查询参数。例如，**q: str = None** 表示 **q** 是一个可选的字符串类型查询参数，默认值为 **None**。

启动应用和测试路由

使用 Uvicorn 启动应用：

```
uvicorn main:app --reload
```

访问 **http://127.0.0.1:8000** 查看根路径的响应：

FastAPI 教程

```
1 // 20231219103752
2 // http://127.0.0.1:8000/
3
4 ▼ {
5     "Hello": "World"
6 }
```

访问 <http://127.0.0.1:8000/items/42?q=runoob> 查看带路径参数和查询参数的响应：

```
1 // 20231219105520
2 // http://127.0.0.1:8000/items/5?q=runoob
3
4 ▼ {
5     "item_id": 5,
6     "q": "runoob"
7 }
```

FastAPI 自动生成的交互式 API 文档将包括定义的路由信息、路径参数、查询参数等。访问文档地址 <http://127.0.0.1:8000/docs> 查看详细的文档和测试界面：

Fast API - Swagger UI

127.0.0.1:8000/docs

Fast API

0.1.0 OAS3

/openapi.json

default

⌵

GET

/items/{item_id}

Read Item Get

Parameters

Try it out

Name	Description
item_id ★ required integer (path)	
q string (query)	

Responses

Code	Description	Links
200	Successful Response application/json Controls Accept header.	No links
422	Validation Error application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string"] }] }</pre>	No links

FastAPI 请求和响应

在 FastAPI 中，请求（Request）和响应（Response）是与客户端交互的核心。

FastAPI 提供了强大的工具来解析请求数据，并根据需要生成规范的响应。

接下来我们来详细看下 FastAPI 的请求和响应。

HTTP 相关内容可以参考：[HTTP 请求方法](#)。

请求数据

查询参数

以下实例中我们定义了一个 `/items/` 路由，接受两个查询参数 `skip` 和 `limit`，它们的类型均为整数，默认值分别为 `0` 和 `10`。

实例

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/items/")
```

```
def read_item(skip: int = 0, limit: int = 10):
```

```
    return {"skip": skip, "limit": limit}
```

在命令行中运行以下命令以启动应用：

```
uvicorn main:app --reload
```

现在，打开浏览器并访问 `http://127.0.0.1:8000/items/`，返回了默认的 JSON 数据：

```
1 // 20231221151237
2 // http://127.0.0.1:8000/items/
3
4 ▼ {
5     "skip": 0,
6     "limit": 10
7 }
```

传递 GET 请求的参数 `http://127.0.0.1:8000/items/?skip=1&limit=5`，返回 JSON 数据如下所示：

```
1 // 20231221151453
2 // http://127.0.0.1:8000/items/?skip=1&limit=5
3
4 ▼ {
5     "skip": 1,
6     "limit": 5
7 }
```

路径参数

FastAPI 教程

我们可以把参数设置在路径上，这样 URL 看起来更美观一些。

以下实例我们定义了一个带有路径参数 `item_id` 和查询参数 `q` 的路由。

实例

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int, q: str = None):
```

```
    return {"item_id": item_id, "q": q}
```

传递 GET 请求的参数 `http://127.0.0.1:8000/items/5?q=runoob`，返回 JSON 数据如下所示：

```
1 // 20231221151952
```

```
2 // http://127.0.0.1:8000/items/5?q=runoob
```

```
3
```

```
4 {
```

```
5     "item_id": 5,
```

```
6     "q": "runoob"
```

```
7 }
```

5 是路径上的参数

请求体

接下来我们创建了一个 `/items/` 路由，使用 `@app.post` 装饰器表示这是一个处理 POST 请求的路由。

实例

```
from pydantic import BaseModel
```

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
class Item(BaseModel):
```

```
    name: str
```

```
    description: str = None
```

```
    price: float
```

```
    tax: float = None
```

```
@app.post("/items/")
```

```
def create_item(item: Item):
```

```
    return item
```

使用 Pydantic 模型 `Item` 定义了一个请求体，包含多个字段，其中一些有默认值，更多 Pydantic 介绍参考：[FastAPI Pydantic 模型](#)。

接下来我们可以打开 `http://127.0.0.1:8000/docs` 来进行 POST 测试：

填写请求参数：

POST **/items/** Create Item

Parameters

No parameters

Request body required

```
{
  "name": "runoob",
  "description": "菜鸟教程 POST 测试",
  "price": 12.0,
  "tax": 1.0
}
```

Execute

返回结果如下：

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/items/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "runoob",
    "description": "菜鸟教程 POST 测试",
    "price": 12.0,
    "tax": 1.0
  }'
```

Request URL

<http://127.0.0.1:8000/items/>

Server response

Code	Details
200	Response body <pre>{ "name": "runoob", "description": "菜鸟教程 POST 测试", "price": 12, "tax": 1 }</pre> Response headers <pre>content-length: 81 content-type: application/json date: Thu, 21 Dec 2023 07:26:47 GMT server: uvicorn</pre>

Responses

响应数据

FastAPI 教程

返回 JSON 数据

路由处理函数返回一个字典，该字典将被 FastAPI 自动转换为 JSON 格式，并作为响应发送给客户端：

实例

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/items/")
```

```
def read_item(skip: int = 0, limit: int = 10):
```

```
    return {"skip": skip, "limit": limit}
```

以上代码在浏览器访问 <http://127.0.0.1:8000/items/>，返回了 JSON 数据：

```
1 // 20231221151237
2 // http://127.0.0.1:8000/items/
3
4 ▼ {
5     "skip": 0,
6     "limit": 10
7 }
```

返回 Pydantic 模型

路由处理函数返回一个 Pydantic 模型实例，FastAPI 将自动将其转换为 JSON 格式，并作为响应发送给客户端：

实例

```
from pydantic import BaseModel
```

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
class Item(BaseModel):
```

```
    name: str
```

```
    description: str = None
```

```
    price: float
```

```
    tax: float = None
```

```
@app.post("/items/")
```

```
def create_item(item: Item):
```

```
    return item
```

POST 请求，返回的数据格式如下所示：

```
{
  "name": "runoob",
  "description": "菜鸟教程 POST 测试",
  "price": 12,
  "tax": 1
}
```

FastAPI 教程

请求头和 Cookie

使用 `Header` 和 `Cookie` 类型注解获取请求头和 `Cookie` 数据。

实例

```
from fastapi import Header, Cookie
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/items/")
```

```
def read_item(user_agent: str = Header(None), session_token: str = Cookie(None)):
    return {"User-Agent": user_agent, "Session-Token": session_token}
```

以上代码在浏览器访问 <http://127.0.0.1:8000/items/>，返回了 JSON 数据：

```
1 // 20231221153857
2 // http://127.0.0.1:8000/items/
3
4 {
5     "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X
6     10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
7     Chrome/120.0.0.0 Safari/537.36",
8     "Session-Token": null
9 }
```

重定向和状态码

使用 `RedirectResponse` 实现重定向，将客户端重定向到 `/items/` 路由。

实例

```
from fastapi import Header, Cookie
from fastapi import FastAPI
from fastapi.responses import RedirectResponse
```

```
app = FastAPI()
```

```
@app.get("/items/")
```

```
def read_item(user_agent: str = Header(None), session_token: str = Cookie(None)):
    return {"User-Agent": user_agent, "Session-Token": session_token}
```

```
@app.get("/redirect")
```

```
def redirect():
    return RedirectResponse(url="/items/")
```

以上代码在浏览器访问 <http://127.0.0.1:8000/redirect/> 会自动跳转到 <http://127.0.0.1:8000/items/> 页面：

FastAPI 教程

```
1 // 20231221153857
2 // http://127.0.0.1:8000/items/
3
4 {
5     "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X
6     10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
7     Chrome/120.0.0.0 Safari/537.36",
8     "Session-Token": null
9 }
```

使用 `HTTPException` 抛出异常，返回自定义的状态码和详细信息。

以下实例在 `item_id` 为 42 会返回 404 状态码：

实例

```
from fastapi import HTTPException
```

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int):
```

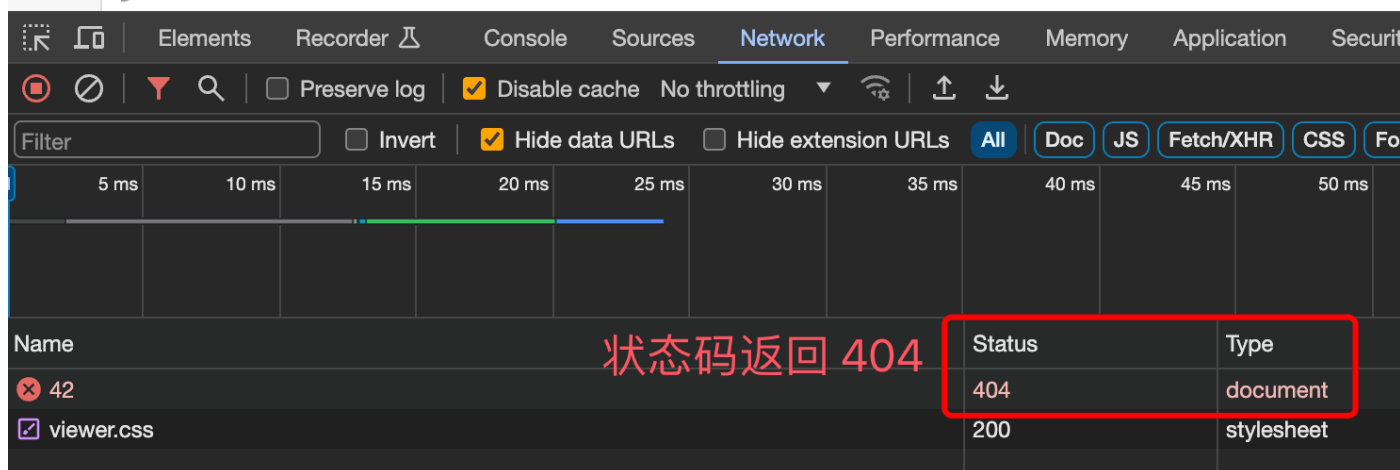
```
    if item_id == 42:
```

```
        raise HTTPException(status_code=404, detail="Item not found")
```

```
    return {"item_id": item_id}
```

以上代码在浏览器访问 `http://127.0.0.1:8000/items/42/` 页面显示如下：

```
1 // 20231221154321
2 // http://127.0.0.1:8000/items/42
3
4 {
5     "detail": "Item not found"
6 }
```



自定义响应头

使用 `JSONResponse` 自定义响应头：

实例

```
from fastapi import FastAPI
```

```
from fastapi.responses import JSONResponse
```

FastAPI 教程

app = FastAPI()

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int):
```

```
    content = {"item_id": item_id}
```

```
    headers = {"X-Custom-Header": "custom-header-value"}
```

```
    return JSONResponse(content=content, headers=headers)
```

以上代码在浏览器访问 <http://127.0.0.1:8000/items/42/> 页面显示如下，可以看到我们自定义的响应头：

```
1 // 20231221154622
2 // http://127.0.0.1:8000/items/42
3
4 {
5     "item_id": 42
6 }
```

The screenshot shows the Chrome DevTools Network tab. The 'viewer.css' resource is selected, showing a 42 status code. The 'Response Headers' section is expanded, displaying the following headers:

Header	Value
Content-Length	14
Content-Type	application/json
Date	Thu, 21 Dec 2023 07:46:22 GMT
Server	uvicorn
X-Custom-Header	custom-header-value

FastAPI 教程

FastAPI Pydantic 模型

Pydantic 是一个用于数据验证和序列化的 Python 模型库。

它在 FastAPI 中广泛使用, 用于定义请求体、响应体和其他数据模型, 提供了强大的类型检查和自动文档生成功能。

以下是关于 Pydantic 模型的详细介绍:

1. 定义 Pydantic 模型

使用 Pydantic 定义一个模型非常简单, 只需创建一个继承自 `pydantic.BaseModel` 的类, 并在其中定义字段。字段的类型可以是任何有效的 Python 类型, 也可以是 Pydantic 内置的类型。

实例

```
from pydantic import BaseModel
```

```
class Item(BaseModel):
```

```
    name: str
```

```
    description: str = None
```

```
    price: float
```

```
    tax: float = None
```

以上代码中, 我们定义了一个名为 `Item` 的 Pydantic 模型, 包含了四个字段: `name`、`description`、`price` 和 `tax`, `name` 和 `price` 是必需的字段, 而 `description` 和 `tax` 是可选的字段, 其默认值为 `None`。

2. 使用 Pydantic 模型

2.1 请求体验证

在 FastAPI 中, 可以将 Pydantic 模型用作请求体 (Request Body), 以自动验证和解析客户端发送的数据。

实例

```
from fastapi import FastAPI
```

```
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Item(BaseModel):
```

```
    name: str
```

```
    description: str = None
```

```
    price: float
```

```
    tax: float = None
```

```
@app.post("/items/")
```

```
def create_item(item: Item):
```

```
    return item
```

以上代码中, `create_item` 路由处理函数接受一个名为 `item` 的参数, 其类型是 `Item` 模型。FastAPI 将自动验证传入的 JSON 数据是否符合模型的定义, 并将其转换为 `Item` 类型的实例。

2.2 查询参数验证

Pydantic 模型还可以用于验证查询参数、路径参数等。

实例

```
from fastapi import FastAPI, Query
```

```
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Item(BaseModel):
```

```
    name: str
```

```
    description: str = None
```

FastAPI 教程

price: float

tax: float = None

```
@app.get("/items/")
```

```
def read_item(item: Item, q: str = Query(..., max_length=10)):
```

```
    return {"item": item, "q": q}
```

以上代码中，`read_item` 路由处理函数接受一个 `Item` 模型的实例作为查询参数，以及一个名为 `q` 的字符串查询参数。通过使用 `Query` 函数，我们还可以为查询参数指定更多的验证规则，如最大长度限制。

3. 自动文档生成

使用 `Pydantic` 模型的一个重要优势是，它能够自动为 `FastAPI` 生成交互式 API 文档。文档会包括模型的字段、类型、验证规则等信息，让开发者和 API 使用者能够清晰地了解如何正确使用 API。

打开 <http://127.0.0.1:8000/docs>，API 文档显示如下：

The screenshot shows the FastAPI interactive API documentation for the `GET /items/` endpoint. The interface includes a header with the method `GET` and the endpoint `/items/` with the description `Read Item`. Below this is a 'Parameters' section with a 'Try it out' button. A table lists the parameters, showing a required query parameter `q` of type `string` with a maximum length of 10. A text input field for `q` is provided. The 'Request body' section indicates it is required and the media type is `application/json`. At the bottom, the 'Example Value' and 'Schema' are shown, displaying a JSON object with fields `name`, `description`, `price`, and `tax`.

Name	Description
<code>q</code> * required	
<code>string</code>	
<code>(query)</code>	
<code>maxLength: 10</code>	

Request body required application/json

Example Value | Schema

```
{
  "name": "string",
  "description": "string",
  "price": 0,
  "tax": 0
}
```

4. 数据转换和序列化

`Pydantic` 模型不仅提供了验证功能，还可以用于将数据转换为特定类型（例如 `JSON`）或反向序列化。在 `FastAPI` 中，这种能力是自动的，你无需手动处理。

通过使用 `Pydantic` 模型，你可以更轻松地定义和验证数据，使得代码更清晰、更健壮，并通过自动生成的文档提供更好的 API 交互体验。

接下来我们可以打开 <http://127.0.0.1:8000/docs> 来进行 `POST` 测试：

填写请求参数：

POST

/items/ Create Item

Parameters

No parameters

Request body required

```
{
  "name": "runoob",
  "description": "菜鸟教程 POST 测试",
  "price": 12.0,
  "tax": 1.0
}
```

Execute

返回结果如下：

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/items/' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "name": "runoob",
  "description": "菜鸟教程 POST 测试",
  "price": 12.0,
  "tax": 1.0
}'
```

Request URL

```
http://127.0.0.1:8000/items/
```

Server response

Code

Details

200

Response body

```
{
  "name": "runoob",
  "description": "菜鸟教程 POST 测试",
  "price": 12,
  "tax": 1
}
```

Download

Response headers

```
content-length: 81
content-type: application/json
date: Thu, 21 Dec 2023 07:26:47 GMT
server: uvicorn
```

Responses

FastAPI 路径操作依赖项

FastAPI 提供了简单易用，但功能强大的依赖注入系统，这个依赖系统设计的简单易用，可以让开发人员轻松地把组件集成至 FastAPI。

FastAPI 提供了路径操作依赖项（Path Operation Dependencies）的机制，允许你在路由处理函数执行之前或之后运行一些额外的逻辑。

依赖项就是一个函数，且可以使用与路径操作函数相同的参数。

路径操作依赖项提供了一种灵活的方式来组织代码、验证输入、进行身份验证等。

接下来我们会具体介绍 FastAPI 路径操作依赖项的相关知识点。

1、依赖项（Dependencies）

依赖项是在路由操作函数执行前或后运行的可复用的函数或对象。

它们被用于执行一些通用的逻辑，如验证、身份验证、数据库连接等。在 FastAPI 中，依赖项通常用于两个方面：

- **预处理（Before）依赖项：** 在路由操作函数执行前运行，用于预处理输入数据，验证请求等。
- **后处理（After）依赖项：** 在路由操作函数执行后运行，用于执行一些后处理逻辑，如日志记录、清理等。

1.1 依赖注入

依赖注入是将依赖项注入到路由操作函数中的过程。

在 FastAPI 中，通过在路由操作函数参数中声明依赖项来实现依赖注入。

FastAPI 将负责解析依赖项的参数，并确保在执行路由操作函数之前将其传递给函数。

1.2 依赖项的使用

定义依赖项：

实例

```
from fastapi import Depends, FastAPI
```

```
app = FastAPI()
```

```
# 依赖项函数
```

```
def common_parameters(q: str = None, skip: int = 0, limit: int = 100):  
    return {"q": q, "skip": skip, "limit": limit}
```

在这个例子中，`common_parameters` 是一个依赖项函数，用于预处理查询参数。

在路由中使用依赖项：

实例

```
from fastapi import Depends
```

```
# 路由操作函数
```

```
@app.get("/items/")
```

```
async def read_items(common: dict = Depends(common_parameters)):  
    return common
```

在这个例子中，`read_items` 路由操作函数中的参数 `common` 使用了 `Depends(common_parameters)`，表示 `common_parameters` 是一个依赖项。FastAPI 将在执行路由操作函数之前运行 `common_parameters` 函数，并将其返回的结果传递给 `read_items` 函数。

2、路径操作依赖项的基本使用

2.1 预处理（Before）

以下实例中，`common_parameters` 是一个依赖项函数，它接受查询参数 `q`、`skip` 和 `limit`，并返回一个包含这些参数的字典。

在路由操作函数 `read_items` 中，通过传入 `Depends(common_parameters)`，我们使用了这个依赖项函数，实现了在路由执行前预处理输入数据的功能。

实例

FastAPI 教程

```
from fastapi import Depends, FastAPI, HTTPException
```

```
app = FastAPI()
```

```
# 依赖项函数
```

```
def common_parameters(q: str = None, skip: int = 0, limit: int = 100):  
    return {"q": q, "skip": skip, "limit": limit}
```

```
# 路由操作函数
```

```
@app.get("/items/")
```

```
async def read_items(common: dict = Depends(common_parameters)):  
    return common
```

2.2 后处理 (After)

以下例子中，**after_request** 是一个后处理函数，用于在路由执行后执行一些逻辑。

在路由操作函数 `read_items_after` 中，通过传入 `Depends(after_request)`，我们使用了这个后处理依赖项，实现了在路由执行后进行额外操作的功能。

实例

```
from fastapi import Depends, FastAPI, HTTPException
```

```
app = FastAPI()
```

```
# 依赖项函数
```

```
def common_parameters(q: str = None, skip: int = 0, limit: int = 100):  
    return {"q": q, "skip": skip, "limit": limit}
```

```
# 路由操作函数
```

```
@app.get("/items/")
```

```
async def read_items(common: dict = Depends(common_parameters)):  
    return common
```

```
# 后处理函数
```

```
async def after_request():
```

```
    # 这里可以执行一些后处理逻辑，比如记录日志
```

```
    pass
```

```
# 后处理依赖项
```

```
@app.get("/items/", response_model=dict)
```

```
async def read_items_after(request: dict = Depends(after_request)):  
    return {"message": "Items returned successfully"}
```

3、多个依赖项的组合

以下例子中，**common_parameters** 和 **verify_token** 是两个不同的依赖项函数，**verify_token** 依赖于 **common_parameters**，这种组合依赖项的方式允许我们在路由执行前先验证一些参数，然后在进行身份验证。

实例

```
from fastapi import Depends, FastAPI, HTTPException
```

```
app = FastAPI()
```

```
# 依赖项函数 1
```

FastAPI 教程

```
def common_parameters(q: str = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

# 依赖项函数 2
def verify_token(token: str = Depends(common_parameters)):
    if token is None:
        raise HTTPException(status_code=400, detail="Token required")
    return token

# 路由操作函数
@app.get("/items/")
async def read_items(token: dict = Depends(verify_token)):
    return token
```

4、异步依赖项

依赖项函数和后处理函数可以是异步的，允许在它们内部执行异步操作。
以下例子中，**get_token** 是一个异步的依赖项函数，模拟了一个异步操作。
在路由操作函数 `read_items` 中，我们使用了这个异步依赖项函数。

实例

```
from fastapi import Depends, FastAPI, HTTPException
from typing import Optional
import asyncio

app = FastAPI()

# 异步依赖项函数
async def get_token():
    # 模拟异步操作
    await asyncio.sleep(2)
    return "fake-token"

# 异步路由操作函数
@app.get("/items/")
async def read_items(token: Optional[str] = Depends(get_token)):
    return {"token": token}
```

通过使用路径操作依赖项，你可以在路由执行前或后执行额外的逻辑，从而实现更灵活、可组合的代码组织方式。

FastAPI 表单数据

在 FastAPI 中，接收表单数据是一种常见的操作，通常用于处理用户通过 HTML 表单提交的数据。FastAPI 提供了 Form 类型，可以用于声明和验证表单数据。

1、声明表单数据模型

接下来我们设计一个接收一个登陆的表单数据，要使用表单，需预先安装 python-multipart:

pip install python-multipart。

代码如下：

实例

```
from fastapi import FastAPI, Form
```

```
app = FastAPI()
```

```
@app.post("/login/")
```

```
async def login(username: str = Form(), password: str = Form()):
```

```
    return {"username": username}
```

接下来我们可以进入 API 文档 <http://127.0.0.1:8000/docs> 进行测验：

The screenshot displays the FastAPI interactive API documentation. At the top, a form for the `/login/` endpoint is shown, with fields for `username` (value: `runoob`) and `password` (value: `123456`). Both fields are marked as required. Below the form is a blue `Execute` button. Underneath the form, the word `表单` (Form) is written in red. The `Responses` section below shows the curl command for a POST request to `http://127.0.0.1:8000/login/`. The server response is shown as a 200 status code with a JSON body: `{"username": "runoob"}`. A red arrow points to this response body, with the text `返回数据` (Return data) written in red next to it.

```
username * required
string
runoob

password * required
string
123456
```

表单

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/login/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'username=runoob&password=123456'
```

Request URL

```
http://127.0.0.1:8000/login/
```

Server response

Code	Details
200	<pre>{ "username": "runoob" }</pre>

返回数据

FastAPI 教程

使用 Pydantic 模型来声明表单数据模型。

在模型中，使用 Field 类型声明每个表单字段，并添加必要的验证规则。

实例

```
from pydantic import BaseModel, Field
```

```
class Item(BaseModel):
    name: str = Field(..., title="Item Name", max_length=100)
    description: str = Field(None, title="Item Description", max_length=255)
    price: float = Field(..., title="Item Price", gt=0)
```

以上例子中，Item 是一个 Pydantic 模型，用于表示表单数据。

模型中的字段 name、description 和 price 分别对应表单中的不同输入项，并设置了相应的验证规则。

除了可以在 API 文档中测验，另外我们也可以自己创建 html 来测试：

实例

```
<form action="http://localhost:8000/items/" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>
  <br>
  <label for="description">Description:</label>
  <textarea id="description" name="description"></textarea>
  <br>
  <label for="price">Price:</label>
  <input type="number" id="price" name="price" required min="0">
  <br>
  <button type="submit">Submit</button>
</form>
```

2、在路由中接收表单数据

在路由操作函数中，可以使用 Form 类型来接收表单数据。

Form 类型的参数可以与 Pydantic 模型的字段一一对应，以实现表单数据的验证和转换。

实例

```
from fastapi import FastAPI, Form
```

```
app = FastAPI()
```

```
# 路由操作函数
```

```
@app.post("/items/")
```

```
async def create_item(
```

```
    name: str = Form(...),
```

```
    description: str = Form(None),
```

```
    price: float = Form(..., gt=0),
```

```
):
```

```
    return {"name": name, "description": description, "price": price}
```

以上例子中，create_item 路由操作函数接收了三个表单字段：name、description 和 price，这些字段与 Item 模型的相应字段一致，FastAPI 将自动根据验证规则验证表单数据。

接下来我们可以进入 API 文档 <http://127.0.0.1:8000/docs> 进行测验：

FastAPI 教程

Request body required

name * required

string

runoob

description

string

coding 学习

☐ Send empty value

price * required

number

12.00

表单

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/items/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'name=runoob&description=coding%20%E5%AD%A6%E4%B9%A0&price=12.00'
```

Request URL

<http://127.0.0.1:8000/items/>

Server response

Code

Details

200

Response body

```
{
  "name": "runoob",
  "description": "coding 学习",
  "price": 12
}
```

返回值

Response headers

```
content-length: 60
content-type: application/json
date: Fri, 22 Dec 2023 10:07:38 GMT
server: uvicorn
```

3、表单数据的验证和文档生成

使用 Pydantic 模型和 Form 类型，表单数据的验证和文档生成都是自动的。

FastAPI 将根据模型中的字段信息生成交互式 API 文档，并根据验证规则进行数据验证。

API 文档地址 <http://127.0.0.1:8000/docs>。

default

POST /items/ Create Item

Parameters

No parameters

Request body *required*

application/x-www-form-urlencoded

name *required*
string

description
string

Try it out

default

POST /items/ Create Item

Parameters

No parameters

Request body *required*

application/x-www-form-urlencoded

name *required*
string

description
string

☒ Send empty value

price *required*
number

price

Execute

Responses

4、处理文件上传

如果表单包含文件上传，可以使用 `UploadFile` 类型处理。

以下是一个处理文件上传的实例：

实例

```
from fastapi import FastAPI, File, UploadFile
```

```
app = FastAPI()
```

```
# 路由操作函数
```

```
@app.post("/files/")
```

```
async def create_file(file: UploadFile = File(...)):
```

```
    return {"filename": file.filename}
```

在这个例子中，`create_file` 路由操作函数接收了一个 `UploadFile` 类型的文件参数。

FastAPI 将负责处理文件上传，并将文件的相关信息包装在 `UploadFile` 对象中，可以轻松地获取文件名、内容类型等信息。

通过上述方式，FastAPI 提供了一种简单而强大的方法来接收和处理表单数据，同时保持了代码的清晰性和可维护性。